OpsPilot AI: The Ultimate Execution Blueprint for Hackathon Domination

SECTION 1 — EXECUTIVE REALITY CHECK

The conceptual architecture of OpsPilot AI identifies a profound systemic inefficiency within modern enterprise architecture: the fragmentation of operational data across disparate, multimodal silos.[1] Organizations operate in environments where critical intelligence is scattered across structured telemetry databases and highly unstructured artifacts, including PDF invoices, customer complaint emails, and photographic evidence of physical assets.[1] The fundamental idea is to autonomously ingest this multimodal chaos, mathematically correlate it, and output a visually coherent, explicitly explainable evidence graph that pinpoint operational anomalies.[1]

The core value proposition is not simply data aggregation; it is explainable cross-modal operational reasoning.[1] The overarching Artificial Intelligence for IT Operations (AIOps) sector is forecast to surge to $37.79 billion by 2031, yet existing solutions are crippled by user skepticism regarding "black box" algorithms that take action without transparent justification.[1] Furthermore, the logistics sector loses over $1 billion annually to overage, shortage, or damage (OS&D) issues, primarily because claims are rejected due to misfiled or poorly correlated multimodal documentation.[1] OpsPilot AI targets this exact pain point, creating unique value by combining the diagnostic power of multimodal entity linking (MEL) with an unyielding commitment to deterministic audit trails.

However, evaluating the provided product specification reveals a concept with exceptional enterprise potential but severe, fatal feature bloat for a time-constrained hackathon environment. The original product vision mandates user authentication, live incident feeds, historical predictive analytics, complex Slack and Gmail integrations, and multi-user operational dashboards.[1] If a student team attempts to build this entire scope over a weekend, the project will inevitably collapse into an unfinished, non-demonstrable state. Furthermore, the proposed technological approach of utilizing Tesseract OCR and EasyOCR for foundational content extraction represents a catastrophic technical risk.[1] Traditional optical character recognition engines are notoriously brittle when confronted with visually rich documents, systematically destroying tabular layouts and spatial relationships, which guarantees total failure in downstream semantic retrieval pipelines.[1]

To succeed, the project requires ruthless simplification. The platform absolutely must execute a flawless multimodal correlation pipeline: ingesting three specific files, extracting structural and visual entities, and rendering them as a dynamic, interactive React Flow graph accompanied by an explainability panel.[1] This is realistically buildable in a hackathon setting, provided the team ignores all secondary Software-as-a-Service (SaaS) features. The highest demo impact with the lowest engineering complexity comes entirely from the transformation of raw, disconnected files into an explainable, visual intelligence graph.[1] The visual generation of this graph proves the system's intelligence without requiring complex backend infrastructure to

support thousands of concurrent users.

Feature Category	PRD Vision	Reality Check & MVP Scope	Justification for Simplification
Document Parsing	Tesseract / EasyOCR	IBM Docling (Layout-aware)	Standard OCR destroys tables. Docling preserves layout, which is mandatory for invoice parsing.[1]
User Management	Authentication, RBAC, Profiles	Hardcoded "Admin" state	Auth provides zero "wow factor" and consumes hours of development time.
Integration Layer	Slack, Gmail, Webhooks	Local File Upload Only	External API integrations introduce severe latency and unreliability during live demos.[1]
Analytics Engine	Predictive AI, historical trends	Single-incident visual graph	Training predictive models requires massive datasets and time unavailable in a hackathon.[1]
Explainability	LLM Chain of Thought	Deterministic Audit Trails	Standard LLM prompting hallucinates logic. The system must output exact input hashes and feature constraints.[1]

SECTION 2 — WINNING MVP STRATEGY

Designing the perfect Minimum Viable Product requires engineering the exact user journey the hackathon judges will experience, optimizing exclusively for perceived technical sophistication,

emotional impact, and absolute stability.

The ideal MVP strips away all peripheral features and focuses entirely on a single, highly orchestrated "hero flow." The minimum features required to impress judges include a seamless multimodal file upload zone, real-time visual feedback indicating backend layout parsing, the dynamic generation of a React Flow node graph, and a slide-out explainability panel detailing the algorithmic confidence score.[1] Conversely, features that must not be built include user dashboards, historical search functionality, live data stream ingestion, and cloud-based active directory integrations.

To ensure maximum speed of execution and demonstration reliability, significant portions of the backend infrastructure must be intelligently simulated. While the document parsing and large language model inference must functionally execute in real-time to prove technical legitimacy, the underlying vector database search should be artificially constrained. Attempting to deploy a massive database populated with millions of mock operational logs will induce latency.[1] Instead, the vector space must be constrained to a handful of curated mock records, ensuring millisecond retrieval times and eliminating the risk of cloud timeout errors during the pitch.

The hero flow for demo day must be executed with surgical precision. The narrative opens by anchoring the technology in the universally recognized pain of the operational "war room"—the high-stress environment where teams frantically sift through scattered logs and emails to identify the root cause of an anomaly.[1] The presenter introduces an active incident: a damaged pallet costing the company thousands of dollars. The user journey begins by dragging a complex PDF invoice, a high-resolution photograph of physical damage, and an unstructured customer complaint email into the OpsPilot AI dashboard.[1]

The system transitions into an active loading state, displaying highly technical, pseudo-terminal outputs such as "Executing layout-aware parsing via Docling" and "Fusing multimodal semantic gaps." This explicitly signals to the judges that the platform is not merely wrapping an API around a chat interface. Upon completion, the interface instantly renders a dark-themed, interactive React Flow canvas.[4] The nodes—representing the invoice amount, the shipping ID, and the photographic evidence—animate into position, explicitly connected by red anomaly edges.[5] The emotional impact peaks when the presenter clicks the central correlation node, revealing the Explainability Panel. The system does not merely state that an anomaly exists; it outputs a deterministic audit trail proving that the correlation achieved a 94.2% confidence score because the extracted text in the email matched the physical damage classification in the image.[1]

MVP Priority Level	Feature / Component	Engineering Strategy	Demo Impact Factor
Critical (Must Build)	Multimodal Ingestion UI	Next.js drag-and-drop zone handling	High. Initiates the user journey.

		multipart/form-data.[6]	
Critical (Must Build)	Layout-Aware Parsing	IBM Docling processing PDF to structured Markdown.[7]	High. Proves data integrity is maintained.
Critical (Must Build)	Evidence Graph Canvas	React Flow utilizing custom NodeType arrays.[8]	Maximum. Provides the core visual "wow factor."
Critical (Must Build)	Explainability Panel	UI drawer displaying confidence scores and match logic.[1]	Maximum. Validates enterprise trustworthiness.
Simulated (Fake it)	Vector Database Scale	Pre-seed Qdrant with only 10 mock operational logs.[9]	Neutral. Judges cannot see database size.
Discard (Do Not Build)	Slack/Gmail Integration	Removed from scope entirely.[1]	Negative. High risk of breaking during demo.

SECTION 3 — FULL SYSTEM ARCHITECTURE

The architectural blueprint must navigate the severe constraints of a hackathon: zero financial budget, no enterprise infrastructure, and limited execution time. The proposed architecture leverages lightweight, asynchronous microservices and highly generous free-tier cloud providers.

The frontend architecture is built upon Next.js utilizing the App Router. This framework is essential because it allows for secure Server Actions, masking API keys from the client while providing a robust, highly responsive user interface. For the visual intelligence layer, the architecture integrates @xyflow/react (React Flow), an advanced library for rendering node-based diagrams.[10] React Flow is critical because building a physics-based, interactive graph using raw D3.js or HTML Canvas would consume the entire hackathon duration. By defining a rigid ReactFlowJsonObject encompassing nodes and edges, the frontend can seamlessly map complex backend analytics into an interactive visual state.[8]

The backend architecture requires a structural departure from standard synchronous Node.js servers. Processing multimodal embeddings and layout-aware document parsing is computationally heavy and time-consuming. Standard serverless environments, such as

Vercel's free tier, enforce a strict 10-second timeout, which will immediately kill any long-running artificial intelligence requests. Therefore, the backend must be constructed using Python and FastAPI.[11] FastAPI natively supports asynchronous execution and provides a BackgroundTasks utility.[12] When the Next.js frontend uploads the files, the FastAPI endpoint immediately accepts the payload, returns an HTTP 202 status to the client, and offloads the heavy processing to an isolated background thread, completely bypassing frontend timeout constraints.[13]

For document processing, the architecture integrates IBM Docling as a localized dependency within the backend container. Docling is an open-source, layout-aware parser that excels at converting complex PDFs—including tables and nested forms—into structurally intact Markdown.[2] This is essential; if the structure of an invoice is lost, the subsequent vectorization is rendered useless. By executing Docling locally, the team avoids the heavy costs and strict rate limits associated with proprietary document parsing APIs like LlamaParse.[14]

The AI orchestration and multimodal inference layer relies on the Google Gemini 2.0 Flash model via the Google AI Studio API. While deploying local vision-language models like ColPali offers theoretical advantages [15], they require massive GPU VRAM allocations that are financially inaccessible to student teams. Gemini 2.0 Flash provides a massive 1-million token context window and is natively multimodal, capable of processing images and text simultaneously.[16] Crucially, the 2026 free tier offers 15 Requests Per Minute (RPM) and 1,500 Requests Per Day (RPD), an exceptionally generous allowance that easily accommodates heavy testing and live demonstration requirements without incurring costs.[16]

Data persistence and object storage are handled by Supabase. The 2026 free tier provides 500 MB of PostgreSQL database storage and 1 GB of S3-compatible file storage.[18] Supabase is essential because it eliminates the need to configure separate Amazon S3 buckets and RDS instances, unifying relational data and blob storage behind a single, highly intuitive SDK. For semantic search and evidence correlation, Qdrant Cloud serves as the vector database. Qdrant provides a permanent free tier encompassing 0.5 vCPU, 1GB RAM, and 4GB disk space, capable of storing approximately 1 million vectors.[20] Qdrant will store the dense vector embeddings generated from the parsed Markdown and unstructured emails, enabling high-speed cosine similarity searches.[9]

Architectural Component	Essential/Optional	Technology Choice	Free Tier Constraints	Implementation Complexity
Frontend UI/Graph	Essential	Next.js + React Flow	Vercel Free Tier (100GB bandwidth).	Moderate. Requires precise JSON mapping.[8]

Backend API	Essential	FastAPI + BackgroundTasks	Hugging Face Spaces (Basic CPU Tier).[22]	Moderate. Requires async task handling.[13]
Document Parser	Essential	IBM Docling (Local)	Bounded by container RAM. 0$ API cost.[2]	High. Container dependency management.[2] [3]
AI Inference Engine	Essential	Gemini 2.0 Flash API	15 RPM / 1,500 RPD.[16]	Low. Simple API integration.
Relational Database	Optional (MVP)	Supabase Postgres	500 MB DB size. Pauses after 1 week.[19]	Low. Excellent documentation.
Object Storage	Essential	Supabase Storage	1 GB total size, 50MB per file upload limit.[18]	Low. Handled via Supabase SDK.
Vector Database	Essential	Qdrant Cloud	1GB RAM, 4GB disk, ~1M vectors.[20]	Moderate. Requires embedding dimension alignment.

SECTION 4 — TECHNOLOGY STACK DECISIONS

Choosing the correct technology stack determines the velocity of execution. The selected stack is optimized for maximum reliability, absolute lowest cost, and rapid development, expressly avoiding enterprise monoliths that require dedicated DevOps teams to configure.

The Fastest to Build Stack (Recommended for Hackathons)

The primary recommendation utilizes frameworks that possess massive open-source ecosystems, extensive documentation, and zero-configuration cloud deployment pipelines.

- **Frontend Framework:** Next.js 15 (App Router). Selected over raw React or Vue.js because it provides built-in API routes, seamless Vercel deployment, and optimized image rendering.
- **Styling:** Tailwind CSS combined with shadcn/ui. Writing custom CSS is a catastrophic waste of hackathon time. Shadcn/ui provides pre-built, accessible, enterprise-grade components (modals, slide-out drawers, tables) that can be copy-pasted directly into the

repository, ensuring a premium aesthetic instantly.

- **Graph Visualization:** @xyflow/react (React Flow). Building SVG node graphs from scratch using D3.js requires advanced mathematics and days of debugging. React Flow provides drag-and-drop mechanics, bezier curve edge rendering, and customizable node types out of the box.[3]
- **Backend Framework:** Python FastAPI. Selected over Express.js or Django. Python dominates the AI/ML landscape, and FastAPI's native support for Pydantic validation and asynchronous UploadFile processing makes it the undisputed best choice for handling complex data ingestion pipelines.[6]
- **Document Parsing:** docling. A clear victor over proprietary APIs like LlamaParse. While LlamaParse possesses a strict 10,000 credit free-tier limit that will be exhausted during extensive testing [14], Docling is open-source, runs locally, and excels at preserving tabular data structures.[2]
- **AI Model Provider:** Google Gemini 2.0 Flash. Competitors like OpenAI GPT-4o lack generous free tiers. Gemini 2.0 Flash is natively multimodal, processes 1 million tokens, and provides 15 Requests Per Minute at zero cost.[16]
- **Database and Storage:** Supabase. Selected over Neon Postgres. While Neon offers excellent scale-to-zero serverless PostgreSQL with 0.5GB storage [27], it lacks an integrated object storage solution. Supabase provides both relational tables and a 1GB file storage bucket, eliminating the need to provision AWS S3 and manage complex IAM policies.[19]
- **Vector Database:** Qdrant Cloud. Selected over Pinecone or local FAISS. Local FAISS instances lose their index upon container restarts, which is highly problematic during hackathons. Qdrant provides a permanent, managed cloud endpoint for free, ensuring vector persistence.[9]
- **Hosting and Deployment:** Vercel (Frontend) and Hugging Face Spaces (Backend). Standard backend providers like Render enforce aggressive 15-minute sleep cycles on free tiers, causing 30-second cold starts that destroy live demos.[11] Hugging Face Spaces provides a continuous, free "CPU Basic" Docker environment, keeping the FastAPI server permanently awake.[22]

Component Requirement	The Recommended Stack	The Failed Alternative	Reason Alternative Was Rejected
Backend Execution	FastAPI on Hugging Face	Node.js on Vercel Functions	Vercel enforces a 10-second timeout on free-tier serverless functions; AI

			parsing takes longer, causing 504 Gateway Errors.
Multimodal Intelligence	Google Gemini 2.0 Flash	Local ColPali + GPU	ColPali requires significant VRAM and expensive GPU instances.[30] Gemini is free and accessible via API.[16]
Document Processing	IBM Docling	PyPDF2 / Tesseract	Legacy OCR destroys layout context, rendering tabular invoice data useless for semantic search.[1]
Vector Storage	Qdrant Cloud	Local FAISS implementation	Ephemeral containers wipe local memory. Qdrant provides a persistent, free managed cloud endpoint.[9]

SECTION 5 — IMPLEMENTATION ROADMAP

Execution requires an unyielding, sequential roadmap. Building features out of order will result in integration chaos and blocked developers. The following represents a high-intensity 48-hour hackathon build sequence.

PHASE 0 — Setup and Scaffolding (Hours 0-4)

The objective is to establish communication between all cloud environments before writing complex business logic.

- **Tasks:** Initialize the Next.js repository and deploy the empty shell to Vercel. Initialize the FastAPI Python environment, write a basic /health endpoint, and deploy to Hugging Face Spaces via Docker.[22] Provision Supabase and Qdrant Cloud projects, extracting and distributing API keys to local .env files.[31]
- **Dependencies:** None.
- **Estimated Time:** 4 hours.
- **Risks:** Hugging Face Docker build failures due to environment mismatches.[11]

- **Fallback Plan:** Build and run the FastAPI Dockerfile locally on a team member's machine to isolate dependency errors before pushing to the cloud proxy.[23]

PHASE 1 — The Ingestion Pipeline (Hours 4-12)

The objective is to successfully move files from the user interface into the backend parsing engine.

- **Tasks:** Implement the Next.js drag-and-drop file uploader. Construct the FastAPI endpoint accepting list[UploadFile].[25] Integrate the Supabase SDK to save the raw files to the storage bucket.[19] Integrate IBM Docling within a BackgroundTasks function to parse the uploaded PDF and return structured Markdown.[7]
- **Dependencies:** Phase 0 completion.
- **Estimated Time:** 8 hours.
- **Risks:** Docling dependencies exhausting the Hugging Face CPU tier memory limit.[23]
- **Fallback Plan:** If layout-aware parsing breaks the container, immediately downgrade to a basic PyPDF text extraction script. The accuracy will drop, but the pipeline will remain functional for the demo.

PHASE 2 — Multimodal AI Integrations (Hours 12-24)

This is the intellectual core of the project. The objective is to establish mathematical correlations between disparate files.

- **Tasks:** Implement the Gemini 2.0 Flash API.[32] Construct a massive prompt that passes the Docling Markdown (invoice), the raw email text, and the image to the model simultaneously. Instruct the model to perform Generative Multimodal Entity Linking (GEMEL), extracting shipment IDs and anomaly descriptors.[33] Generate embeddings for these entities and insert them into Qdrant Cloud.[15]
- **Dependencies:** Successful execution of Phase 1 parsing.
- **Estimated Time:** 12 hours.
- **Risks:** Hitting the 15 RPM Gemini API limit during aggressive testing.[17]
- **Fallback Plan:** Rotate API keys across multiple developer Google accounts to bypass rate limits.

PHASE 3 — MVP Implementation and Graph Mapping (Hours 24-34)

The objective is to visualize the intelligence generated in Phase 2.

- **Tasks:** Agree on a rigid JSON schema with the backend team. Map the backend correlation response to the ReactFlowJsonObject structure, populating the nodes and edges arrays.[8] Construct custom node components (TextUpdaterNode, ImageNode).[3] Implement SVG bezier curves connecting the nodes based on the AI's confidence scores.[5]
- **Dependencies:** Phase 2 backend JSON structure.
- **Estimated Time:** 10 hours.

PHASE 4 — Polish and Explainability (Hours 34-40)

The objective is to elevate the prototype from functional to enterprise-grade.

- **Tasks:** Design the slide-out Explainability Panel. Format the deterministic audit trail, displaying the exact input hashes, extracted strings, and the AI confidence threshold.[1] Apply strict dark-mode styling and Framer Motion layout transitions. Ensure the UI handles loading states gracefully with terminal-style text readouts.
- **Estimated Time:** 6 hours.

PHASE 5 & 6 — Deployment and Demo Optimization (Hours 40-48)

The objective is absolute live-demo stability.

- **Tasks:** Curate the exact three "Golden Path" mock files used for the pitch. Hardcode backend fallback logic to guarantee the correct JSON response if the AI API times out. Rehearse the 3-minute pitch, timing the presentation perfectly with the React Flow animation sequence.
- **Estimated Time:** 8 hours.
- **Sacrifice if behind schedule:** Abandon dynamic Qdrant vector retrieval entirely. Have FastAPI pass the three files directly to Gemini, ask for a correlation JSON, and pipe that directly to the frontend.

SECTION 6 — TEAM ROLE BREAKDOWN

Executing complex multimodal architectures within 48 hours requires a highly disciplined, deeply parallelized team structure. Assuming a four-person student team, responsibilities must be strictly siloed to prevent merge conflicts and blocked workflows.

1. The Graph Architect (Frontend Engineer)

- *Responsibilities:* Next.js initialization, Tailwind CSS / shadcn integration, React Flow canvas implementation, custom node rendering, and Framer Motion animation sequencing.[10]
- *Task Management Strategy:* This role must never be blocked by backend development. During Phase 0, the Graph Architect negotiates a strict mock JSON contract with the backend team (the exact structure of the nodes and edges arrays).[8] For the first 30 hours, they build the entire visual graph using a static, local JSON file.

2. The Ingestion Engineer (Backend / DevOps)

- *Responsibilities:* FastAPI routing, managing asynchronous BackgroundTasks, Hugging Face Spaces Docker deployment, Supabase bucket integration, and IBM Docling layout parsing.[7]
- *Task Management Strategy:* Focuses purely on data plumbing. Ensures that files uploaded on the frontend are reliably received, stored, and structurally converted into machine-readable Markdown without crashing the server.[13]

3. The Intelligence Engineer (AI/ML Integrator)

- *Responsibilities:* Gemini 2.0 Flash prompt engineering, Generative Multimodal Entity Linking (GEMEL) execution, Qdrant Cloud vector insertion, and mathematical similarity scoring.[15]

- *Task Management Strategy:* Acts as the bridge between raw data and visual intelligence. Takes the parsed Markdown from the Ingestion Engineer, feeds it through the LLM alongside the raw images, and outputs the exact JSON structure required by the Graph Architect.

4. The Strategist (Product Lead & Pitch Master)

- *Responsibilities:* Crafting the demonstration narrative, sourcing the highly specific "Golden Path" mock data (the broken pallet image, the PDF invoice), designing the user interface layout, and managing the Git repository to resolve conflicts.
- *Task Management Strategy:* Shields the engineers from scope creep and distraction. Constantly audits the build against the "Hero Flow" requirements.

To prevent integration chaos, the team must utilize a trunk-based development Git workflow. Feature branches should be short-lived (maximum 4 hours). Massive, sprawling branches that attempt to build the entire backend before merging will result in catastrophic merge conflicts at hour 40.

SECTION 7 — AI & AUTOMATION STRATEGY

The AI strategy must aggressively combat the stereotype of being "just another AI wrapper" by demonstrating explicit, deterministic operational reasoning across multiple modalities.
Standard Retrieval-Augmented Generation (RAG) is insufficient for this project. Traditional RAG pipelines chunk text blindly, embed it, and perform cosine similarity searches, which fails completely when attempting to link unstructured text to visual damage in an image.[1] Instead, the architecture must implement a Generative Multimodal Entity Linking (GEMEL) strategy.[35]
The Gemini 2.0 Flash model is specifically chosen because it allows the massive context window (1 million tokens) to act as an orchestration engine that analyzes text, visual, and tabular data simultaneously.[16]
The prompt engineering strategy must enforce rigid constraints. The prompt will inject the Docling-extracted Markdown (representing the invoice), the raw text of the customer complaint email, and the binary payload of the damaged freight photograph. The model is instructed to:

1. Extract absolute identifiers (e.g., Invoice Numbers, Shipment IDs).
2. Analyze the image to classify the physical damage (e.g., "crushed corner," "water damage").
3. Determine if the textual complaint matches the visual classification.
4. Output the response *strictly* as a structured JSON object containing a mathematically scaled confidence score and a deterministic reasoning string.

To reduce inference costs and bypass rate limits, the system does not poll the AI continuously. The architecture operates entirely on event-driven batch processing—the AI is only triggered when the specific three-file bundle is uploaded by the user.[6]
Hallucination prevention is paramount. The primary risk of utilizing large language models for operational intelligence is that they will fabricate correlations to appease the prompt.[1] To mitigate this, the system enforces response validation logic. If the AI returns a correlation confidence score below 90%, the backend mathematically forces the relationship edge to be severed, flagging the anomaly as requiring "Human Review."

During the demonstration, the judges will care most about the Explainability Panel. Providing a generic summary such as "The AI thinks these are related" is a massive failure. The AI must explicitly output the extracted entities it used to make the decision, effectively simulating a deterministic audit trail that satisfies enterprise compliance mandates.[1]

SECTION 8 — DEPLOYMENT & DEVOPS PLAN

The deployment strategy is optimized for simplicity, zero cost, and zero DevOps overhead, ensuring the platform remains highly accessible throughout the judging period.

The Next.js frontend will be deployed exclusively on Vercel. Vercel provides seamless GitHub integration; every push to the main branch automatically triggers a production rebuild. Environment variables, such as the Hugging Face backend API URL, are injected securely via the Vercel dashboard, ensuring API keys are never exposed to the client.

The FastAPI backend poses a significant deployment challenge. Standard free-tier hosting providers, such as Render, implement aggressive scaling policies, forcing applications to sleep after 15 minutes of inactivity.[11] Waking these servers requires up to 30 seconds, which will catastrophically derail a live 3-minute pitch. Therefore, the backend must be deployed using the Hugging Face Spaces Docker SDK.[22] Hugging Face provides a continuous, free "CPU Basic" environment that prevents cold starts.

The Dockerfile for Hugging Face must be meticulously configured. It must expose port 7860 and run the Uvicorn ASGI server binding to 0.0.0.0.[11] Because the IBM Docling parser and associated machine learning models are massive dependencies, the Dockerfile must be configured to utilize Hugging Face cache directories (/tmp) to prevent image build timeouts.[23]

Database hosting relies entirely on the managed Supabase instance. The PostgreSQL connection string and secure API keys are stored as secrets within the Hugging Face environment variables. Supabase handles connection pooling natively, ensuring the FastAPI backend does not overwhelm the database with concurrent connections.[19] Qdrant Cloud handles the vector index management, providing a stable REST endpoint for embedding insertions and similarity searches.[36]

This architecture completely eliminates the need for complex CI/CD orchestration tools like Jenkins or GitHub Actions. The separation of concerns—Vercel for the client, Hugging Face for the processor, Supabase for state, and Qdrant for vectors—creates a highly resilient, distributed free-tier ecosystem.

SECTION 9 — FAILURE PREVENTION

Hackathon demonstrations occur under extreme pressure, utilizing overloaded venue Wi-Fi and tightly constrained free-tier limits. A battle-tested execution strategy assumes that every external dependency will eventually fail and builds paranoid mitigation systems accordingly.

Failure Mode	Root Cause Analysis	Emergency Mitigation & Backup Plan

API Quota Exhaustion	Reaching the 15 RPM limit on Gemini 2.0 Flash during aggressive final-hour testing.[16]	Hardcode an array of 5 backup Google API keys in the .env file. Implement round-robin key rotation logic in the FastAPI client.
Serverless Timeout Failure	Vercel aborts HTTP requests exceeding 10 seconds. AI parsing routinely takes 15+ seconds.	Never await the AI response synchronously. FastAPI returns an immediate HTTP 202 status; Vercel implements a setInterval polling loop to fetch results when ready.[12]
Container Memory Crash	IBM Docling layout extraction exhausts the 16GB RAM limit on Hugging Face Spaces.[23]	Implement a lightweight fallback script using PyPDF2 that triggers automatically if the primary Docling module raises an Out-Of-Memory (OOM) exception.
Visual Graph Hairball	The AI correlates too many entities, rendering the React Flow canvas a chaotic, unreadable mess.[1]	Frontend logic hard-caps the rendered graph to a maximum of 7 nodes. Additional edges are hidden behind a progressive disclosure "Expand Context" button.
Complete Cloud Blackout	Venue Wi-Fi fails entirely, severing access to Vercel, Hugging Face, and Supabase.	**The Offline Vault:** The team must maintain a perfectly functional version of the app running on localhost:3000 and localhost:8000, pre-seeded with mock JSON responses.

The most common UX confusion occurs during latency periods. If the user uploads documents and the screen remains static for 10 seconds, credibility is destroyed. To prevent this, the frontend must implement highly specific, dynamic loading states. As the backend executes its

asynchronous tasks, the frontend should display pseudo-terminal logs: *"Initializing Docling Layout Engine...", "Generating Multimodal Embeddings...", "Calculating Semantic Proximity...".* This transforms perceived latency into an exhibition of deep computational sophistication.

SECTION 10 — DEMO DOMINATION STRATEGY

To secure victory, the presentation must actively manipulate judge psychology. Evaluators are subjected to dozens of generic AI applications that merely wrap an API around a chat box. OpsPilot AI must establish an immediate, visceral connection to a massive financial problem, followed rapidly by a visually stunning technological resolution that proves profound engineering depth.[1]

The 3-Minute Demo Structure

0:00 - 0:30: The Hook & The Pain
Opening Line: "Last month, a major logistics client lost $140,000 on a single shipment dispute simply because the photographic evidence, the invoice, and the customer complaint were trapped in disconnected silos."
Storytelling: The presenter explains that manually resolving these discrepancies takes weeks, and existing AIOps tools demand perfectly mapped databases to fix it. "We built a system that skips the database entirely."
0:30 - 1:30: The Live Execution
Action: The presenter drags the three highly complex mock files onto the OpsPilot AI dashboard.
Script: "OpsPilot AI is an explainable multimodal correlation engine. Watch the terminal output. We are bypassing standard OCR and utilizing IBM Docling to mathematically preserve the tabular layout of this invoice, while simultaneously passing the visual freight damage through a Generative Multimodal Entity Linking pipeline. We aren't just summarizing text; we are structurally fusing modalities."
1:30 - 2:30: The Visual Reveal & Explainability *Action:* The React Flow graph violently renders, connecting the disparate nodes.[4] The presenter clicks the central anomaly node to reveal the Explainability Panel. *Script:* "This is the Evidence Graph. In seconds, the AI mapped the physical damage directly to the financial record. But enterprise operations cannot trust a black box. If I click this correlation, the Explainability Engine provides a deterministic audit trail—showing the exact input hash, the extracted entity IDs, and the 94.2% confidence score. OpsPilot AI doesn't guess; it mathematically proves exactly why a failure occurred."
2:30 - 3:00: The Close
Ending Line: "We executed this using Next.js, FastAPI, IBM Docling, and Gemini 2.0 Flash orchestration. OpsPilot AI transforms fragmented chaos into auditable, enterprise-grade intelligence. Thank you."

Maximizing Judge Psychology Advantage

Teams frequently lose despite building excellent technology because they fail to communicate architectural difficulty. By explicitly mentioning "bypassing standard OCR," "Generative Multimodal Entity Linking," and "deterministic audit trails," the presenter forces the judges to

recognize the project's intellectual rigor. The visual fluidity of the React Flow graph [34], combined with the strict, austere UI design, subliminally communicates that the platform is a stable, enterprise-ready product rather than a fragile student experiment.

SECTION 11 — DESIGN & UX STRATEGY

The application must instantly communicate authority, stability, and high financial value, distancing itself from colorful, consumer-grade applications. The design must emulate the austere precision of a Bloomberg financial terminal or a Palantir enterprise dashboard.[1]

The color strategy mandates a strict dark mode interface. Backgrounds should utilize deep charcoal (#0f172a), accented by muted electric blue (#3b82f6) for neutral edges, and strict crimson (#ef4444) for detected anomalies. Playful gradients, rounded bubble buttons, and bright primary colors must be entirely avoided.

Typography is critical for establishing technical credibility. Standard UI elements should utilize a clean, geometric sans-serif font like Inter or Geist. However, the Explainability Panel and all extracted entity data must utilize a strict monospace font, such as JetBrains Mono or Fira Code. Monospace typography subliminally signals technical precision, auditability, and raw data integrity to the user.

Animations must be utilized, but sparingly. Bouncing or elastic transitions destroy the enterprise facade. Using Framer Motion, nodes within the React Flow canvas should fade and slide up sequentially with a 0.2-second staggered delay, communicating ordered data processing.[34] The SVG edges connecting the nodes should trace themselves utilizing stroke-dasharray animations, visually demonstrating the active "linking" of operational intelligence.[5]

Acceptable design shortcuts include hardcoding the peripheral dashboard statistics (such as "Total Incidents Analyzed: 1,432"). Evaluators will not scrutinize static background numbers if their attention is forcefully directed to the live graph generation. Conversely, a severe UX credibility killer is a frozen user interface. If the frontend completely locks up while awaiting the FastAPI background task, judges will assume the application has crashed. A dynamic, pulsing loading overlay is an absolute necessity.

SECTION 12 — FINAL MASTER PLAN

OpsPilot AI is an exceptionally ambitious concept that targets a massively capitalized market inefficiency. The execution blueprint outlined above provides a rigid, battle-tested pathway to translating this theoretical vision into a dominating hackathon prototype.

- **Final Architecture Recommendation:** Next.js Server Actions $\rightarrow$ FastAPI BackgroundTasks $\rightarrow$ Supabase $\rightarrow$ Qdrant Cloud $\rightarrow$ React Flow Visualization.
- **Final MVP Recommendation:** A highly constrained, three-file upload pipeline focusing exclusively on graph generation and deterministic explainability.[1]
- **Final Stack Recommendation:** Tailwind CSS, shadcn/ui, IBM Docling [2], Gemini 2.0 Flash [16], Vercel, and Hugging Face Spaces.[22]
- **Biggest Success Factor:** The visual fluidity and seamless animation of the React Flow

graph mapping exactly to the complex PDF invoice extraction.
- **Most Likely Reason to Fail:** Docker container memory limitations crashing the FastAPI Hugging Face deployment during the heavy IBM Docling layout parsing execution.[23]

The "Do This / Do Not Do This" Checklist

- **DO:** Focus 80% of all engineering resources purely on the backend parsing pipeline and the frontend React Flow JSON data mapping.
- **DO:** Utilize the Gemini 2.0 Flash free tier to perform Generative Multimodal Entity Linking, leveraging the 1-million token context window to fuse modalities mathematically.[16]
- **DO NOT:** Attempt to build user authentication, database login systems, RBAC, or live webhooks.
- **DO NOT:** Use standard OCR (Tesseract). If Docling proves too heavy for the container, implement the lightest layout-aware parser possible, but avoid tools that permanently destroy tabular arrays.[1]

The Non-Negotiables

1. A functional drag-and-drop file uploader that successfully routes multi-part files to the isolated Python backend.[6]
2. The React Flow Canvas dynamically rendering at least three explicitly connected nodes based on the raw uploaded data.[3]
3. The slide-out Explainability Panel displaying a calculated confidence score and deterministic reasoning.[1]

Cut These First If Time Runs Out

If development velocity stalls, cut the Qdrant Cloud vector database integration entirely. Attempting complex semantic retrieval across disparate indices is difficult to debug under pressure. Instead, have the FastAPI backend directly prompt the Gemini API with the three uploaded files, request a localized correlation JSON object, and pipe that directly to the frontend. Cut the Supabase PostgreSQL database integration and simply store the raw files locally in the /tmp directory of the backend container, wiping them after the graph generates.[23]

Final Evaluation Score

- **Technical Impressiveness:** 9.5/10 (If layout-aware parsing and cross-modal generative entity linking are successfully executed).
- **Execution Realism:** 8.5/10 (Highly feasible due to the combination of generous 2026 API free tiers and decoupled asynchronous background tasks).
- **Hackathon Winning Potential:** 9.5/10 (Directly targets "wrapper fatigue" with high visual impact and an explicitly transparent audit trail).
- **Startup Potential:** 9.0/10 (The lack of deterministic explainability in current AIOps platforms represents a deeply validated, highly lucrative market gap).
- **Implementation Difficulty:** 8.5/10 (Requires rigorous coordination between frontend React state management and backend Python asynchronous data pipelines).

Works cited

1. Startup Idea_ OpsPilot AI Analysis.docx
2. Documentation - Docling - GitHub Pages, accessed on May 28, 2026, https://docling-project.github.io/docling/
3. Custom Nodes - React Flow, accessed on May 28, 2026, https://reactflow.dev/learn/customization/custom-nodes
4. Collaborative Flowchart | Liveblocks Example, accessed on May 28, 2026, https://liveblocks.io/examples/collaborative-flowchart/nextjs-react-flow
5. Custom Edges - React Flow, accessed on May 28, 2026, https://reactflow.dev/learn/customization/custom-edges
6. Request Files - FastAPI, accessed on May 28, 2026, https://fastapi.tiangolo.com/tutorial/request-files/
7. A REST Implementation of 'Docling' with FastAPI - DEV Community, accessed on May 28, 2026, https://dev.to/aairom/a-rest-implementation-of-docling-with-fastapi-ogn
8. ReactFlowJsonObject - React Flow, accessed on May 28, 2026, https://reactflow.dev/api-reference/types/react-flow-json-object
9. Pricing for Cloud and Vector Database Solutions Qdrant, accessed on May 28, 2026, https://qdrant.tech/pricing/
10. Examples - React Flow, accessed on May 28, 2026, https://reactflow.dev/examples
11. Deploying Your FastAPI Applications on Huggingface Via Docker, accessed on May 28, 2026, https://huggingface.co/blog/HemanthSai7/deploy-applications-on-huggingface-spaces
12. Background Tasks - FastAPI, accessed on May 28, 2026, https://fastapi.tiangolo.com/tutorial/background-tasks/
13. FastAPI: combining BackgroundTasks and UploadFile gives SyntaxError: non-default argument follows default argument - Stack Overflow, accessed on May 28, 2026, https://stackoverflow.com/questions/68800405/fastapi-combining-backgroundtasks-and-uploadfile-gives-syntaxerror-non-default
14. LlamaIndex Pricing 2026: Plans, Costs & ROI, accessed on May 28, 2026, https://checkthat.ai/brands/llamaindex/pricing
15. Advanced Retrieval with ColPali & Qdrant Vector Database, accessed on May 28, 2026, https://qdrant.tech/blog/qdrant-colpali/
16. Gemini API Free Tier 2026: Limits, Quotas, and What You Actually Get - PE Collective, accessed on May 28, 2026, https://pecollective.com/tools/gemini-free-tier-guide/
17. Interpreting Google AI Studio Rate Limits 2026 Latest Version: What to Do if Tier 1 RPD 250 is Too Strict, accessed on May 28, 2026, https://help.apiyi.com/en/google-ai-studio-rate-limits-2026-guide-en.html
18. Supabase Free Tier Limits: What You Actually Get In 2026 - AI Agency Plus, accessed on May 28, 2026, https://aiagencyplus.com/supabase-free-tier-limits/

19. Supabase Pricing in 2026: Plans, Free Tier Limits & Full Breakdown | UI Bakery Blog, accessed on May 28, 2026, https://uibakery.io/blog/supabase-pricing
20. Qdrant Cloud Pricing 2026: Tiers, Costs And Self-Hosted Crossover - RankSquire, accessed on May 28, 2026, https://ranksquire.com/2026/04/19/qdrant-cloud-pricing-2026/
21. Qdrant Pricing 2026: Plans, Costs & What You'll Pay - CheckThat.ai, accessed on May 28, 2026, https://checkthat.ai/brands/qdrant/pricing
22. Building AI-Powered APIs with FastAPI and OpenAI Agents SDK: Deployment on Hugging Face | by Daniel Hashmi | Dev Genius, accessed on May 28, 2026, https://blog.devgenius.io/building-ai-powered-apis-with-fastapi-and-openai-agents-sdk-deployment-on-hugging-face-2ce34d3eb766
23. Deploying a FastAPI App on Hugging Face Spaces — and Handling All Its Restrictions | by Nasrin Mazaheri | Medium, accessed on May 28, 2026, https://medium.com/@na.mazaheri/deploying-a-fastapi-app-on-hugging-face-spaces-and-handling-all-its-restrictions-d494d97a78fa
24. Pricing & Fees - Supabase, accessed on May 28, 2026, https://supabase.com/pricing
25. File Uploading and Background Tasks on FastAPI | by Marcelo Eduardo Benencase, accessed on May 28, 2026, https://medium.com/@marcelo.benencase/file-uploading-and-background-tasks-on-fastapi-883d73f5ea61
26. docling-project/docling: Get your documents ready for gen AI - GitHub, accessed on May 28, 2026, https://github.com/docling-project/docling
27. Neon Serverless Postgres Pricing 2026: Complete Breakdown & Cost Comparison - Vela, accessed on May 28, 2026, https://vela.simplyblock.io/articles/neon-serverless-postgres-pricing-2026/
28. Neon Serverless Postgres Guide for TypeScript Backends in 2026 - Encore, accessed on May 28, 2026, https://encore.dev/articles/neon-serverless-postgres
29. Supabase Pricing in 2026: What the Free Tier Actually Gets You - Cotera, accessed on May 28, 2026, https://cotera.co/articles/supabase-pricing-guide
30. ColPali: Efficient Document Retrieval with Vision Language Models - OpenReview, accessed on May 28, 2026, https://openreview.net/forum?id=ogjBpZ8uSi
31. Qdrant Setup, accessed on May 28, 2026, https://qdrant.tech/course/essentials/day-0/qdrant-cloud/
32. accessed on May 28, 2026, https://www.metacto.com/blogs/the-true-cost-of-google-gemini-a-guide-to-api-pricing-and-integration#:~:text=New%20Free%20Tier%20Limits%20(April,Reduced%20quotas%20(varies%20by%20region)
33. Generative Multimodal Entity Linking - LREC, accessed on May 28, 2026, http://www.lrec-conf.org/proceedings/lrec-coling-2024/pdf/2024.main-1.676.pdf
34. FREE Workflow Builder Tutorial - Next.js 16, React Flow, MongoDB | Full Source Code, accessed on May 28, 2026, https://www.youtube.com/watch?v=6ZgR4UajJhQ
35. Generative Multimodal Entity Linking - arXiv, accessed on May 28, 2026, https://arxiv.org/html/2306.12725v1

36. Security - Qdrant, accessed on May 28, 2026, https://qdrant.tech/documentation/security/